Session 7 · Deck 7.B2

Solving the Growth Model as Supervised Learning on the Euler Equation

Zhigang Feng

Thursday, November 5, 2026

© Zhigang Feng Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 7.B2 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6 ✓	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — learning a function: 7.A1 the learning problem and the neuron · 7.A2 activations and approximation · 7.A3 backpropagation and training. **Block B — using it:** 7.B1 PyTorch and a forecast · **7.B2** the growth model · 7.B3 architecture and shape · 7.B4 capacity and order.

© Zhiyuan Feng Redistribution prohibited without permission

Roadmap: Four Parts

The network is the policy, sampled states are the inputs, and the Euler equation supplies the labels.

Part	Topic	The question it answers
1	The model	What is the unknown function, and what condition must it satisfy?
2	How we usually solve it	What does time iteration (3.A3) deliver, and at what cost?
3	Reframing it as supervised learning	Where do the labels come from when there are no data?
4	How we test it	Is the network's answer right where it was not trained?

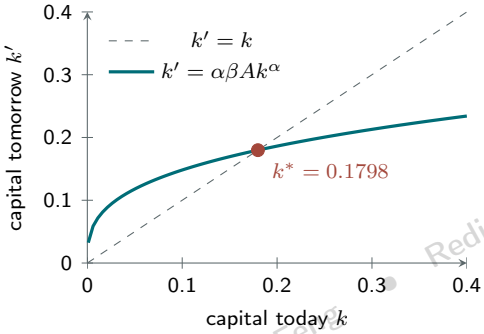
© Jesse Perla · Redistribution prohibited without permission

Idea

5.B1's residual, with 7.A2's learned basis: the Euler residual at sampled states is the prediction error, and zero is the label. This keeps 5.B1's promise that the projection framework survives a basis that is not fixed in advance.

© Jesse Perla · Feng

Solving the model means finding one function, the policy $k' = \pi(k)$.



- Maximise $\sum_t \beta^t u(c_t)$ subject to $c + k' = Ak^\alpha + (1-\delta)k$.
- The answer is a **policy function** $k' = \pi(k)$: capital tomorrow, given capital today.
- Rung 0: log utility, $\delta = 1$, $\beta = 0.96$. The policy is known: $k' = \alpha\beta Ak^\alpha$ (left).
- Rung 1: $u = c^{1-\gamma}/(1-\gamma)$, $\gamma = 2$, $\delta = 0.025$, $\beta = 0.99$ — the quarterly α, β, δ of Sessions 4–5. No closed form; $k^* = 28.35$.
- **Solving the model means finding the function π.**

© Zhigang Feng · Redistribution prohibited without permission

Both rungs: $\alpha = 0.33$, $A = 1$; **the lab's names. Brock & Mirman, *JET* 4(3), 1972; the model of 1.B1 and 3.A3.**

The Euler Equation, and Its Unit-Free Residual

The true policy makes the residual zero at every k.

- The **Euler equation**: at an optimum, giving up one unit of consumption today and eating the proceeds tomorrow must leave utility unchanged,

$$u'(c) = \beta\, u'(c')\, R', \qquad R' = \alpha A k'^{\alpha-1} + 1 - \delta.$$

- Under a candidate policy π, both sides can be computed at any k: $k' = \pi(k)$, c from the budget, then $k'' = \pi(k')$ and c'. Their gap is the **residual** — 3.A1's and 5.B1's.
- Divide and take logs, so the gap has no units:

$$r_\pi(k) = \log \frac{\beta\, u'(c')R'}{u'(c)}.$$

- $r = 0.01$ reads as a one per cent wedge between the two sides, at any calibration.
- **The true policy makes the residual zero at every k.**

© Zhigang Feng · Redistribution prohibited without permission

Time Iteration: the Algorithm

Each round solves the Euler equation exactly at the nodes, given the previous round's policy.

- Grid $k_1 < \cdots < k_N$; resources $f(k) = Ak^\alpha + (1-\delta)k$; a first guess π_0.
- Round n: at each node k_i, solve one equation for today's choice $x = k'$, with tomorrow's behaviour given by last round's policy π_n:

$$u'\big(f(k_i) - x\big) = \beta\, u'\big(f(x) - \pi_n(x)\big)\,\big(\alpha A x^{\alpha-1} + 1 - \delta\big).$$

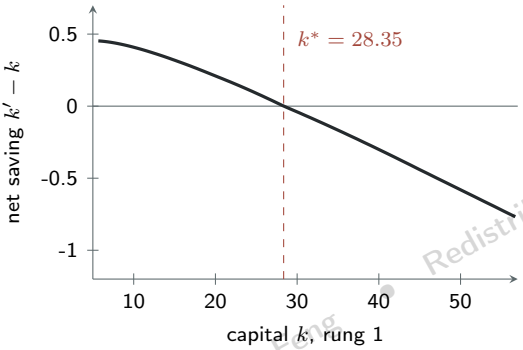
- Set $\pi_{n+1}(k_i) = x_i$. Between nodes, interpolate.
- Stop when $\max_i |\pi_{n+1}(k_i) - \pi_n(k_i)|/\pi_n(k_i) < 10^{-10}$.
- **Each round solves the Euler equation exactly at the nodes, given the previous round's policy** — 3.A3's operator, monotone rather than a contraction.

© Zhigang Feng. Redistribution prohibited without permission

W. Coleman, "Equilibrium in a production economy with an income tax", *Econometrica* 59(4), 1991. Lab 7: 400 nodes, log-spaced; each root by bisection.

Time Iteration's Answer, and Its Cost

Fast and accurate in one dimension. This is the answer the network must match.



- Below k^* the economy saves, above it dissaves: capital moves to the steady state.
- 400 nodes, 292 rounds, 1.2 seconds on one CPU core.
- $\log_{10}$ max Euler error -3.44: one dollar of consumption in about 2,750.
- With d state variables, N nodes per variable become N^d nodes.
- **Fast and accurate in one dimension. This is the answer the network must match.**

© Zhigang Feng · Redistribution prohibited without permission

Lab 7, block D (rung 1).

Solving a Model as Supervised Learning: the Dictionary

Inputs are free: draw as many states as you like, fresh at every step.

Supervised learning	Digits (7.A1–7.A3)	Growth model (this deck)
inputs x_i	images	sampled capital stocks k_i
model f_θ	classifier	the policy $k' = \pi_\theta(k)$
prediction error	$\hat{p}$ against the label	the Euler residual $r_\theta(k_i)$
label	the true digit	**zero**, at every state

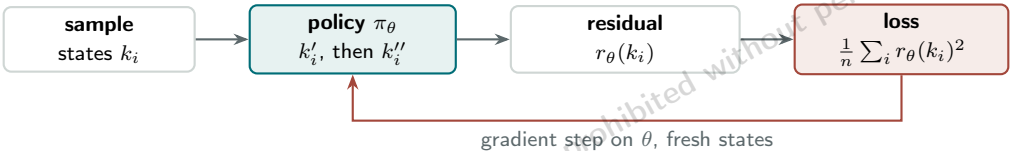
Redistribution prohibited without permission

- Time iteration hands the Euler equation a grid. Here a network is the policy, and the equation becomes the loss.
- Nobody hands us the right k' at each k. The optimality condition says what a right answer must satisfy.
- **Inputs are free: draw as many states as you like, fresh at every step.**

© Zhirang Feng

From Residual to Loss: Zero Is the Label

No benchmark policy ever enters the loss. Benchmarks are kept for testing.

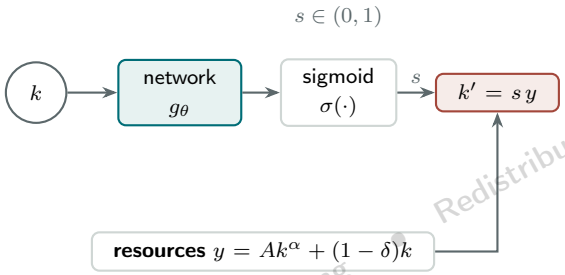


© Thierry Feron · Reproduction prohibited without permission

- Squared error against a label of zero: $\ell_i = (r_\theta(k_i) - 0)^2$. The training loop is 7.B1's five lines.
- The policy is evaluated twice, at k and at k', so gradients flow through both.
- Network: two tanh hidden layers of 32 units; 256 fresh states per step; Adam, 500 steps on rung 0, 3,000 on rung 1.
- **No benchmark policy ever enters the loss. Benchmarks are kept for testing.**

Lab 7, block D. Azinovic, Gaegauf & Scheidegger, "Deep equilibrium nets", *IER* 63(4), 2022.

A constraint written into the architecture cannot be violated, and costs no penalty term.



- **Feasibility**: $c > 0$ and $k' > 0$. One infeasible state makes $\log c$ undefined and ends training.
- The network outputs a *saving share* $s = \sigma(g_\theta) \in (0,1)$, so $k' = s\,y$ and $c = (1-s)\,y$ are positive at every θ.
- The input is $\log k$, rescaled to $[-1,1]$.
- **A constraint written into the architecture cannot be violated, and costs no penalty term.**

© Zhigang Feng Redistribution prohibited without permission

7.B1's five-line loop, with the Euler residual as the loss and fresh states as the batch.

1: $\pi_\theta(k) = f(k)\,\sigma\big(g_\theta(\tilde{k})\big),\ \ \tilde{k} = \log k$ rescaled ▷ feasible
2: start the share at its steady-state value ▷ rung 1
3: **for** step $= 0, \dots, S$ **do**
4: draw 256 states k_i, uniform in $\log k$ on $[0.2, 2]\,k^*$
5: $k_i' \leftarrow \pi_\theta(k_i),\ \ k_i'' \leftarrow \pi_\theta(k_i')$
6: $r_i \leftarrow \log \beta u'(c_i') R_i' - \log u'(c_i)$ ▷ residual
7: $\mathcal{L} \leftarrow \frac{1}{256} \sum_i r_i^2$ ▷ label: zero
8: backward; Adam step on θ ▷ 7.B1
9: **end for**

Against time iteration

No grid, no root-finder, no interpolation: line 4 replaces the grid, and lines 5–8 replace "solve at each node".

The cost of a step does not depend on how many states the model has; it depends on the batch. That is the whole case for the method, and it is not visible in one dimension.

Lab 7, block D; the code is on the next two frames.

© Zhigang Feng
Redistribution prohibited without permission

Feasibility is line 6; the economics is line 10.

```
def saving_share(self, k):
    x = (torch.log(k) - self.log_center) / self.log_scale
    return torch.sigmoid(self.net(x))

def forward(self, k):
    return self.resources(k) * self.saving_share(k)

def log_residual(model, k):
    c, rhs = euler_rhs(model, k)
    return torch.log(rhs) + model.rung.gamma * torch.log(c)

loss = log_residual(model, k).square().mean()
```

- euler_rhs returns c and $\beta u'(c')R'$; with $u'(c) = c^{-\gamma}$, line 10 is $\log$ RHS $- \log u'(c)$.
- **Feasibility is line 6; the economics is line 10.**

© Zheng Tang • Redistribution prohibited without permission

Lab 7, block D, shown without the productivity shock.

The Code: One Training Step

7.B1's five-line loop; only the batch and the loss are new.

```
k, z = sample_states(rung, rung.batch_size)        # fresh states every step
loss = log_residual(model, k, z).square().mean()   # the label is zero
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

- Line 1 is the batch: 256 states drawn fresh, uniform in $\log k$ on the training band. Nothing is reused.
- Line 2 is the loss: the squared log residual, whose label is zero. No benchmark appears anywhere.
- The lab's code carries a productivity shock z: rung 2, Brock–Mirman with a seven-node Gauss–Hermite expectation (4.B2). Lab 7 solves it and passes the same gates.
- **Only the batch and the loss are new; the rest is 7.B1.**

© Zhitang Feng · Redistribution prohibited without permission

Lab 7, block D.

The solution is claimed only where it was trained and tested.

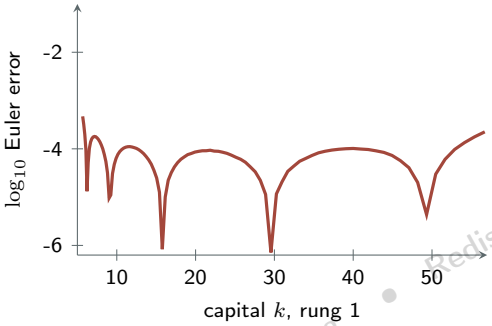


© Zhaoyang Feng · Unauthorized redistribution prohibited without permission

- **Training** states: uniform in $\log k$ on $[0.2,\ 2] \times k^*$, fresh at every step.
- **Held-out** states: new draws from the same band, from a generator training never used. They test coverage.
- **Off-domain** states: $[0.1, 0.2) \cup (2, 3] \times k^*$, never asked of the network. They test extrapolation.
- **The solution is claimed only where it was trained and tested.**

Held-Out and Off-Domain Residuals, on Judd's Scale

Report the maximum, and report it where the network was not trained.



$\log_{10}$ max error	rung 0	rung 1
held-out	-1.95	-3.34
off-domain	-1.29	-1.59

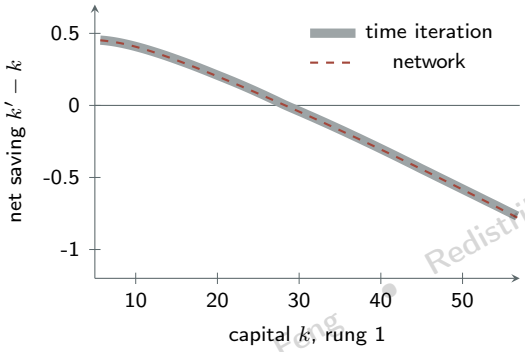
- **Judd's scale**: $\log_{10}$ of the consumption error implied by the residual. -3 is one dollar in a thousand.
- Off-domain, accuracy falls by more than half an order of magnitude on rung 0 and almost two on rung 1.
- **Report the maximum, and report it where the network was not trained.**

© Zhigang Feng · Redistribution prohibited without permission

Lab 7, block D. Judd, *JET* 58(2), 1992.

The Network Against the Known Answer

Neither benchmark entered training, and the two curves lie on top of each other.



- Benchmarks: the closed form (rung 0), time iteration (rung 1). Neither entered training.
- Largest policy error on held-out states: 0.64% on rung 0, 0.04% on rung 1.
- Off-domain: 3.4% and 1.3%.
- On rung 1 the saving share starts at its steady-state value; the lab shows a default start.
- **The two curves lie on top of each other.**

© Zhigang Feng Redistribution prohibited without permission

Lab 7, block D.

The Steady-State Check, and the Gates

A gate is a pass mark fixed before the run. Both rungs pass every gate.

Gate	rung 0	rung 1
feasible at every tested state	yes	yes
max policy error, held-out (gate: below 1%)	0.64%	0.04%
max policy error, off-domain (reported)	3.4%	1.3%
simulated against analytical k^* (gate: 1%)	0.47%	0.18%

© Zhaong Feng Redistribution prohibited without permission

- Steady-state check: from $0.5\,k^*$, apply the learned policy 3,000 times. A *global* test: small errors that compound along the path show up here.
- A **gate** is a pass mark fixed before the run.
- **Both rungs pass every gate.**

Lab 7, block D; policy errors against the closed form (rung 0) and time iteration (rung 1).

4 · Time Iteration and the Network, Side by Side

Learn the method where it loses, so that you can trust it where nothing else runs.

Rung 1	**time iteration**	**network**
numbers stored	400 grid values	1,153 parameters
seconds, one CPU core	1.2 (292 iterations)	4.6 (3,000 steps)
$\log_{10}$ max Euler error, held-out band	-3.44	-3.34

© Jiaqing Fang · Redistribution prohibited without permission

- In one dimension the classical method ties on accuracy and wins on the rest.
- A grid costs N^d; the network's cost grows with the sample. Its case is dimension.
- **Learn the method where it loses, so that you can trust it where nothing else runs.**
- Next, 7.B3: the policy was a plain dense network. What if the architecture itself carried the economics?

Lab 7, block D (rung 1: CRRA $\gamma = 2$, $\delta = 0.025$, $\alpha = 0.33$, $\beta = 0.99$).

Takeaways

- **Part 1.** The unknown is one function, $k' = \pi(k)$, and the Euler equation, in unit-free logs, is the condition it must satisfy at every k.
- **Part 2.** Time iteration solves it at 400 nodes in 292 rounds, to -3.44 on Judd's scale — and pays N^d for a grid.
- **Part 3.** Make the network the policy, sampled states the inputs and the Euler residual the prediction error, with zero as the label; a sigmoid share makes every policy feasible.
- **Part 4.** Test where it was not trained: -3.34 held-out, -1.59 off-domain on rung 1; both rungs pass every gate. In one dimension time iteration ties on accuracy and wins on the rest.

© Zhigang Feng · Redistribution prohibited without permission

Bridge: Put the Economics into the Architecture

The policy was a plain dense network with one restriction, feasibility, written into its last layer. Theory says more than that: saving rises with wealth; a value function is concave in capital.

And the network had more parameters than the problem has grid points. Which of the many fits does training pick, and why does it generalise at all? That is 7.B4's question.

Next: 7.B3

An architecture is a restriction on W. The convolution builds in locality; monotone and input-convex networks guarantee a shape — and each says what it costs.

© Zhigang Feng redistribution prohibited without permission